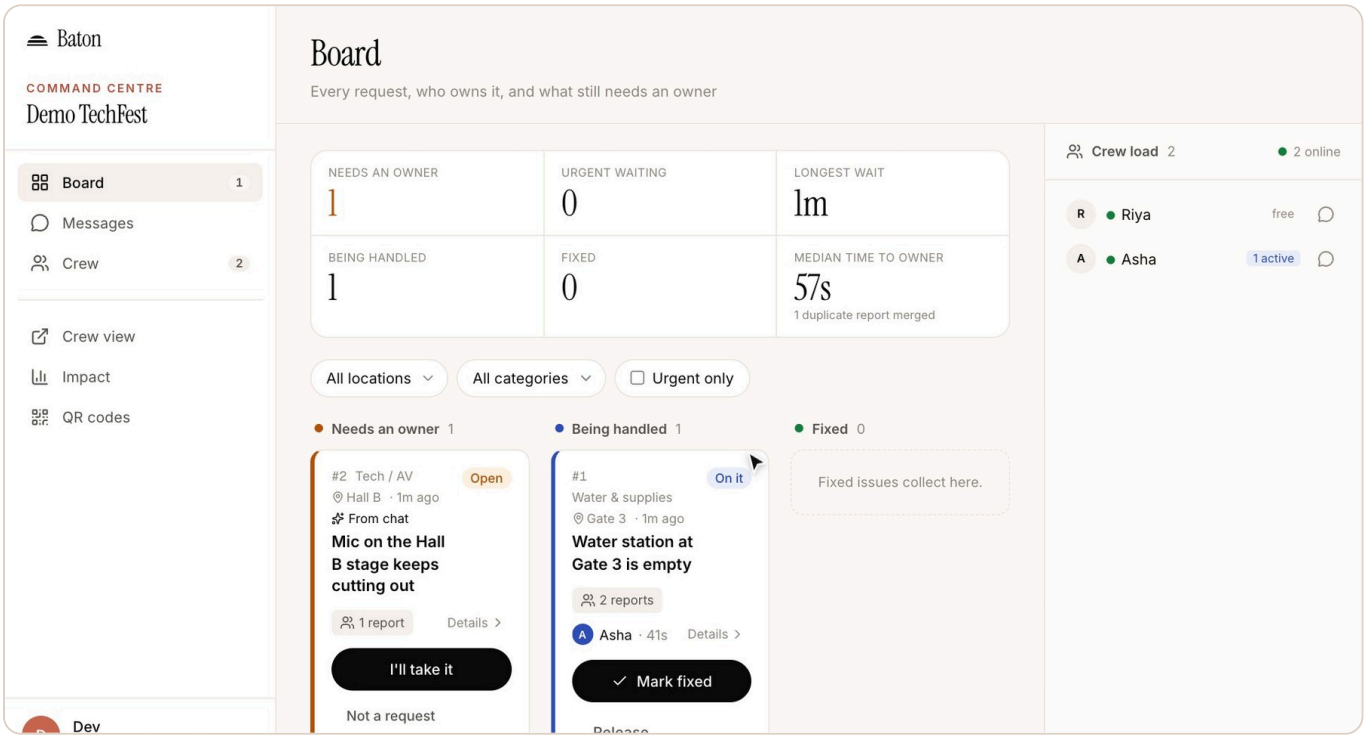
WORKPLACE & COLLABORATION TRACK

Every request has an owner.

Baton is a real-time operations coordination platform for event teams. Attendees report problems by scanning a QR poster. Each report becomes an incident with exactly one owner, visible live on the organizer's board and on every volunteer's phone. Volunteers coordinate in a crew chat, hand work over with an explicit **Baton Pass**, and can start a voice call from the incident. The person who reported the problem sees who is on it and when it is fixed. Chat, live updates, private alerts and calls all run on Nexconn.

LIVE APP	NEXCONN APP KEY	SOURCE CODE	DEMO VIDEO
baton.priyanshnarang.in	e5t4ouvpen0va	github.com/priyansh-narang2308/baton	youtu.be/Cyq-dWpizs0



Organizer board on the live site during the recorded demo run (12 Nov 2026): incidents, owners, crew chat and crew presence, all updated through Nexconn without a reload.

The problem

In an event group chat, everyone sees a request and nobody owns it. Duplicates pile up, handovers are silent, and the person who asked never hears back.

What Baton does

Each request gets one owner, chosen atomically. Duplicate reports merge. Handover only happens when the next person accepts. Every step is timed and logged.

Built on Nexconn

Group and direct channels, custom message types, message metadata, targeted messages, message and presence webhooks, and 1:1 voice calls with the Call SDK.

01 · THE PROBLEM

Event crews run on group chat. Group chat has no owners.

Campus fests, hackathons and meetups are run by small crews of volunteers. On the day, requests come from everywhere: an attendee at a gate, a speaker backstage, another volunteer. Most of it lands in one busy chat group.

That group is good at spreading information. It is bad at getting work done, for four reasons:

- **No owner.** Many people read a request and each assumes someone else has it.
- **Duplicates.** The same issue arrives five times from five people, and the crew cannot tell if it is one problem or five.
- **Silent handovers.** A volunteer leaves for a break or another task and the job quietly drops.
- **No closure.** The person who reported the problem never learns whether anyone came.

REPRESENTATIVE SCENARIO

The water station at Gate 3 runs dry. An attendee tells a volunteer; another posts in the crew group; a third attendee asks at the help desk. Three messages, one problem. Two volunteers each think the other is going. Twenty minutes later the station is still empty and the attendee has walked away.

This is an illustrative scenario built from common event operations, not measured data or user research. It is also the scenario used in the demo.

WHY WE BUILT IT

Ticketing and helpdesk tools solve ownership, but they are built for support teams at desks, not volunteers walking a venue on their phones. Crews will not install an app or create accounts for a one-day event, and attendees certainly will not.

We wanted the speed of a chat group with the accountability of a ticket queue: keep the chat, and add one rule on top of it. **Every request has exactly one owner, and everyone can see who.**

Design principles

- **No accounts on the day.** Volunteers join by QR code; attendees report from a poster with no app.
- **One owner, never zero.** A request is claimed atomically and stays owned during a handover.
- **Chat stays chat.** The crew still talks freely; a request typed in chat becomes a tracked incident.
- **Close the loop.** The reporter sees "Sent", then who is on it, then "Fixed".
- **Coordination, not dispatch.** Baton is for operational requests (supplies, equipment, logistics). It is not an emergency or safety-critical dispatch system.

WHO IT IS FOR

Organizer / ops lead

Sets up the event, prints QR codes, watches the board. Needs to see what is unowned and how long it has waited.

Volunteers

Work from their phones. Claim requests, talk in crew chat, hand over with a Baton Pass, call a lead when stuck.

Attendees

Scan a poster at a location, describe the problem, and see the status update. No account, no install.

02 · THE SOLUTION

From a QR scan to "Fixed", with one owner the whole way.

1 Set up the event

The organizer names the event and its locations (Gate 3, Main stage…). Baton creates a Nexconn crew group and prints a crew QR code plus one report poster per location.

2 Crew joins by QR

Volunteers scan, type a name, and are in the crew chat. No password, no app store.

3 Attendee reports

Scanning a location poster opens a short form. The report is sent and the attendee gets a private status thread.

4 It becomes an incident

A numbered, categorised card appears on the board and on every crew phone. A request typed in crew chat ("water's out at gate 3") becomes one too.

5 Duplicates merge

A second report about the same thing attaches to the existing incident. Matching uses deterministic rules: same location, same category, overlapping words, close in time. No AI.

6 Someone takes it

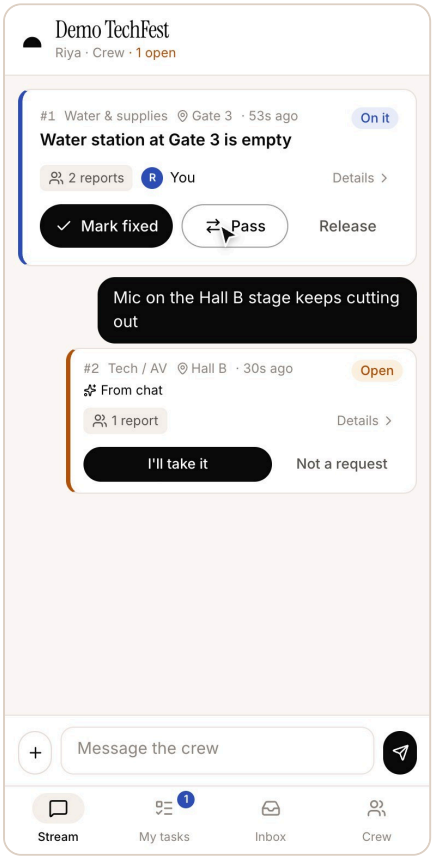
One tap claims the incident. If several volunteers tap at once, exactly one wins; the others are told it is taken.

7 Baton Pass

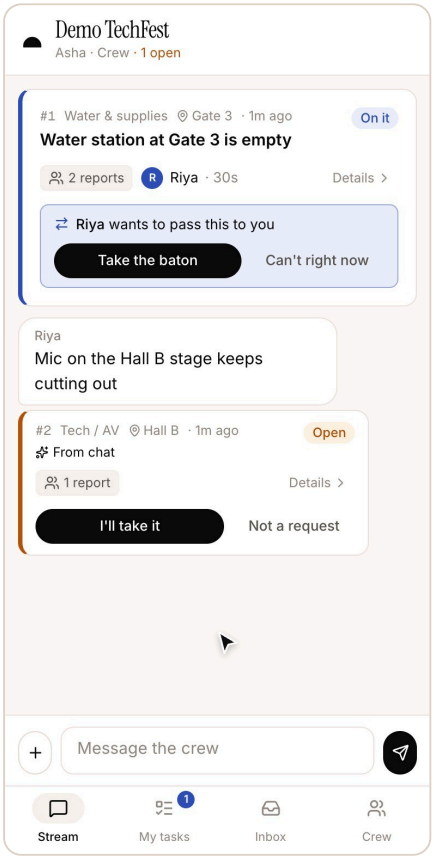
The owner offers the incident to a teammate with a note. They stay the owner until it is accepted, so the job is never ownerless.

8 Call, fix, close the loop

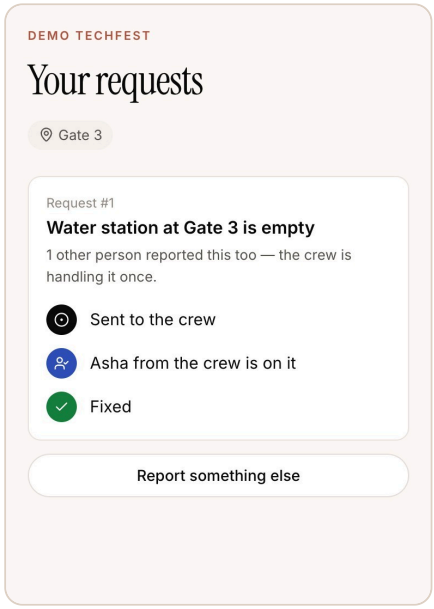
A volunteer can start a voice call from the incident. When it is marked fixed, the reporter sees "Fixed" and the timeline keeps the whole story.



Volunteer. A chat message became a tracked request ("From chat"), now owned by this volunteer.



Baton Pass. Only the recipient gets this private offer. Riya stays owner until "Take the baton" is tapped.



Attendee. The reporter sees who has it, then that it is fixed, through a Nexconn direct message.

Interfaces

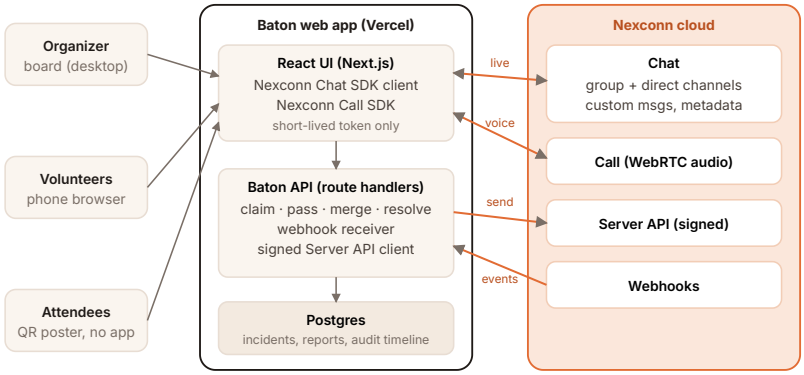
Organizer board (desktop), volunteer view (phone), attendee report page (phone), QR and poster pages. All are custom React screens; the Nexconn Chat UI kit is not used.

03 · NEXCONN INTEGRATION

Nexconn is the real-time layer. Baton is the rules.

Baton uses the **Nexconn Chat SDK** (`@nexconn/chat` 26.2.1) with its own React interface, the **Nexconn Server API** from the backend, **webhooks** for messages and presence, and the **Nexconn Call SDK** (`@nexconn/call`) for voice. The App Secret stays on the server; browsers only receive short-lived access tokens.

BATON FEATURE	NEXCONN CAPABILITY	IMPLEMENTATION ROLE	EVIDENCE
Sign-in without accounts	Server API `/v4/auth/access-token/issue`	Server issues a token per volunteer and per anonymous attendee; client refetches on expiry	LIVE
Crew chat	Group channel (`group-channel/create`, `join`), Chat SDK `GroupChannel.sendMessage`, history query	One group per event; custom React chat UI; late joiners load history	LIVE run check 8
Incident cards in chat	Custom message types `BTN:Incident`, `BTN:Status`, `BTN:Nudge`	Registered with `NCEngine.registerCustomMessages`; the event bot posts cards via the Server API	LIVE check 7
Live owner and status on every screen	Message metadata (`message/metadata/set`, `onMessageMetadataUpdated`)	Card state rides on the message; clients apply only newer versions	LIVE checks 12, 16
Chat line → request	Webhook `message:send`	Signature and freshness checked, envelope deduplicated, rule-based detection, one incident per message	LIVE check 9
Private Baton Pass alert	Group message with `toUserIds`	Only the recipient receives the offer and its push text	LIVE check 14
Reporter updates	Direct channel message (Server API)	"Sent", "<name> is on it", "Fixed", each sent once per reporter	LIVE checks 13, 17, 26
Crew online count	Webhook `user:connection_status`	Presence per member; client IP is dropped before storage	LIVE
Resilient connection	Connection status handler, `NCEngine.connect`	Shows reconnecting state, retries with a fresh token	CODE
Voice huddle	Call SDK: `startCall`, `acceptCall`, `enableMicrophone`, `endCall`	1:1 audio call started from an incident; duration logged on its timeline	LIVE checks 20–24



How the pieces fit

Baton's database is the source of truth for ownership. Nexconn carries every change to every screen: the server updates an incident, then writes the card's metadata and sends targeted messages; clients receive them over their open Chat SDK connection.

Without Nexconn

Baton would need its own WebSocket fleet, message history, offline delivery, push, presence and a WebRTC stack. Nexconn lets the project spend its time on ownership rules instead.

Evidence: "Live" = verified on the production site in the 28-check recorded run of 12 Nov 2026. "Code" = implemented and tested.

04 · WHAT IS NEW, AND HOW IT HOLDS UP

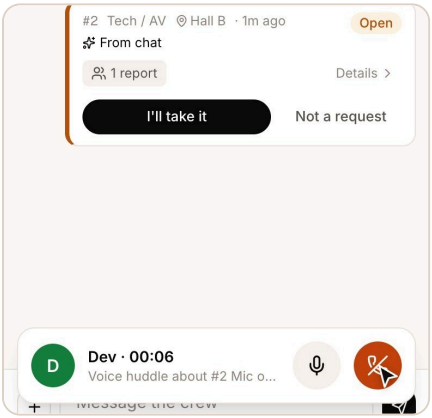
Accountability built into the message layer.

What is different

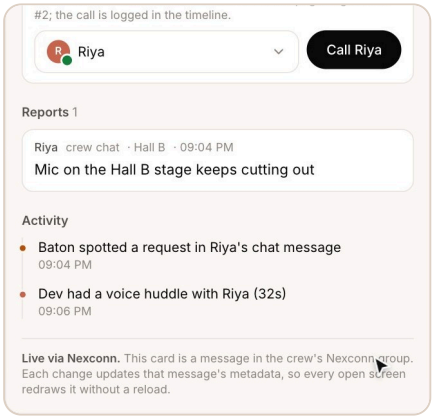
- **The Baton Pass.** Handover is a two-step offer and accept. The current owner keeps the incident until the recipient accepts, so ownership never drops to zero. Offer and accept are written to the audit timeline.
- **Chat as an intake channel.** The crew keeps typing normally. A message webhook spots requests in chat and turns them into incidents, linked back to the message.
- **Live cards, not refreshing lists.** Each incident is a custom Nexconn message whose metadata holds owner and status, so every phone shows the same state.
- **The reporter is in the loop.** Anonymous attendees get their own Nexconn identity and direct-message status updates.
- **A call that belongs to the incident.** The voice huddle starts from the request, and its duration is logged against it.

Correctness under real conditions

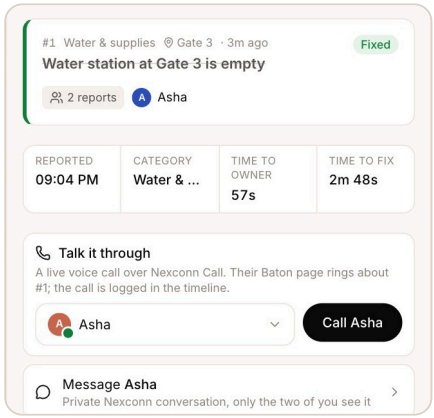
- **Atomic claim.** A conditional database update means five simultaneous claims give one owner and four "already taken" answers (unit-tested).
- **Transactional pass.** Accept checks that the caller is the named recipient and that the incident is still claimed, inside one transaction.
- **Ordered updates.** Metadata carries a version; clients ignore anything older than what they already show.
- **Safe webhooks.** SHA-1 signature and timestamp window, deduplication on the envelope ID, and a unique key on the source message, so retries never double-create.
- **Escalation.** A worker re-alerts unowned incidents, using row locks so two workers never send the same alert.
- **Recovery.** Dropped requests retry quietly; the client reconnects with a fresh token.



Nexconn Call between two crew members, started from an incident.



Incident activity: the chat detection and the 32-second call are both logged.



Fixed incident: two merged reports, time to owner 57 s, time to fix 2 m 48 s.

28 / 28

checks passed in a recorded end-to-end run on the live site, with five separate browser identities. Each check waited for the real result on another person's screen.

102

automated tests in 9 files (claim races, pass rules, merging, webhooks, signatures, projection). Typecheck, lint and production build are clean.

2-way

audio confirmed in the recorded call: each side received the other's speech over Nexconn Call.

Known limitations

Some actions take several seconds on the live site because each makes several database round trips from the server region to the database region; co-locating them is the next fix. Calls are 1:1 audio only. Duplicate matching is rule-based and can miss reports worded very differently; sensitive categories are only suggested as merges, never merged automatically. No read receipts or typing indicators yet.

05 · BUSINESS

Sold to the organizer, used by the crew.

Who pays, who uses

The **buyer** is the organizer or operations lead, who is accountable for how the event runs. The daily **users** are volunteers, and attendees are light, anonymous users. Because nobody but the organizer needs an account, adoption on the day costs the crew one QR scan.

Pricing hypothesis

- **Free** for small events, to remove the first barrier.
- **Per-event plan** for larger one-off events (crew size and locations).
- **Annual plan** for institutions that run many events: colleges, communities, event agencies.

These are hypotheses to be tested in pilots, not validated pricing.

Go-to-market plan

- Start with student-run events: college fests, hackathons, developer meetups and community conferences with 10–150 volunteers.
- Offer a free pilot: set up the event together, print the posters, and review the incident log afterwards.
- Measure what the product claims to change: time to owner, time to fix, unowned requests and duplicate reports.
- If no live event is available in time, run a rehearsal with a real crew and report it as a rehearsal.

Traction: honest status

Baton has **no external users, pilots or customer feedback yet**. All Nexconn users so far are our own development and demo identities. We are not reporting any numbers we do not have.

06 · BUILT DURING THE CHALLENGE

Everything in Baton was built during the challenge window (23 Oct – 22 Nov 2026). The first commit is a Next.js scaffold on 28 Oct 2026; there are 69 commits up to 12 Nov. There was no earlier version of the product.

Third-party building blocks: Next.js, React, Postgres with Drizzle ORM, Framer Motion, Radix UI and Lucide icons; the landing page hero layout is adapted from an Originkit block and the footer from Watermelon's footer-29. The demo narration is synthetic (TTS) and the two voices in the recorded call are synthetic test voices sent through a real Nexconn call.

07 · SUBMISSION DETAILS

NEXCONN APP KEY

`e5t4ouvpen0va`

The App Key used by the live site. The App Secret and all tokens are server-side only and not included in this document.

Project	Baton: the real-time operations coordination platform for event teams
Track	Workplace & Collaboration (team messaging, shift handover)
Live app	https://baton.priyanshnarang.in
Source code	https://github.com/priyansh-narang2308/baton
Demo video	youtu.be/Cyq-dWpizs0
Nexconn products	Chat SDK (custom UI), Server API, webhooks, Call SDK. docs.nexconn.ai
Try it	Open the live app, choose "Set up your event", then open the crew and report QR links on a phone.